

东北大学资源与土木工程学院

作 业 报 告

课程名称: 空间数据库原理

学生专业: 遥感科学与技术

学生班级: 遥感 2301

学 号: 20232444

学生姓名: 闻子博

指导教师: 郭甲腾

作业成绩:

教师评语:

评阅人:

评阅时间:

1、 计算下列坐标点的 Z 曲线索引值

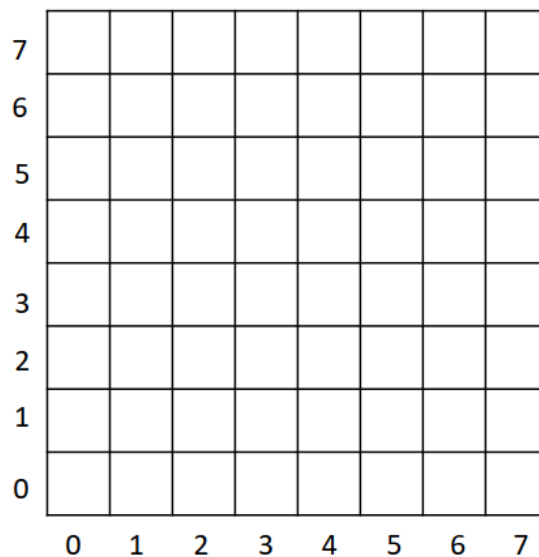
(1,0) , (3,8) , (4,6) , (7,9)

```
def z_curve_index(x, y):  
    def bit_shuffle(a, b, n_bits=32): # 假设坐标值不超 32 位整数范围, 可按需调整  
        result = 0  
        for i in range(n_bits):  
            result |= ((a & (1 << i)) << i) | ((b & (1 << i)) << (i + 1))  
        return result  
    return bit_shuffle(x, y)  
  
points = [(1, 0), (3, 8), (4, 6), (7, 9)]  
for point in points:  
    print(f"坐标 {point} 的 Z 曲线索引值: {z_curve_index(*point)}")
```

问题 输出 调试控制台 终端 端口

```
PS C:\Users\93104> & D:/python/python.exe d:/python/计算下列坐标点的Z曲线索引值.py  
坐标 (1, 0) 的 Z 曲线索引值: 1  
坐标 (3, 8) 的 Z 曲线索引值: 133  
坐标 (4, 6) 的 Z 曲线索引值: 56  
坐标 (7, 9) 的 Z 曲线索引值: 151  
PS C:\Users\93104>
```

2、请计算并绘制下图的3阶Hilbert曲线：



```
import matplotlib.pyplot as plt
import numpy as np

# 设置中文字体支持
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False # 正确显示负号

# 定义 Hilbert 曲线的递归生成函数（保持原有的算法逻辑）
def hilbert_curve(x, y, xi, xj, yi, yj, n, curve_points):
    if n == 0:
        # 计算当前小线段终点坐标并添加到曲线点列表
        X = x + (xi + yi) / 2
        Y = y + (xj + yj) / 2
        curve_points.append((X, Y))
        return
    # 递归绘制 Hilbert 曲线的四个分形部分
    hilbert_curve(x, y, yi / 2, yj / 2, xi / 2, xj / 2, n - 1, curve_points)
    hilbert_curve(x + xi / 2, y + xj / 2, xi / 2, xj / 2, yi / 2, yj / 2, n - 1,
curve_points)
    hilbert_curve(x + xi / 2 + yi / 2, y + xj / 2 + yj / 2, xi / 2, xj / 2, yi /
2, yj / 2, n - 1, curve_points)
    hilbert_curve(x + xi / 2 + yi, y + xj / 2 + yj, -yi / 2, -yj / 2, -xi / 2, -
xj / 2, n - 1, curve_points)

# 生成 Hilbert 曲线的点坐标，并添加平移
def generate_hilbert_curve(order, grid_size=8, translation=0.5):
```

```

"""生成指定阶数的 Hilbert 曲线点，并沿左下 45 度方向平移半个网格单位"""
curve_points = []
# 调整初始参数，使曲线覆盖整个网格并居中
hilbert_curve(0, 0, grid_size, 0, 0, grid_size, order, curve_points)

# 沿左下 45 度方向平移半个网格单位（即 x 和 y 各平移-0.5）
translated_points = [(x - translation, y - translation) for x, y in
curve_points]
    return translated_points

# 创建网格并绘制 Hilbert 曲线
def plot_hilbert_with_grid(order=3, grid_size=8):
    # 生成 Hilbert 曲线点（已平移）
    points = generate_hilbert_curve(order, grid_size)
    x_coords, y_coords = zip(*points)

    # 创建图形
    fig, ax = plt.subplots(figsize=(8, 8))

    # 绘制网格背景（确保坐标系正确）
    ax.set_xlim(-0.5, grid_size - 0.5)
    ax.set_ylim(-0.5, grid_size - 0.5)

    # 设置网格线（在整数位置，使 0-7 坐标位于网格之间）
    for i in range(grid_size + 1):
        ax.axhline(y=i - 0.5, color='gray', linestyle='-', linewidth=0.5)
        ax.axvline(x=i - 0.5, color='gray', linestyle='-', linewidth=0.5)

    # 设置坐标轴标签和标题
    ax.set_xticks(range(grid_size))
    ax.set_yticks(range(grid_size))
    ax.set_xlabel('X 轴', fontsize=12)
    ax.set_ylabel('Y 轴', fontsize=12)
    ax.set_title(f'{order} 阶 Hilbert 曲线', fontsize=14)

    # 设置坐标轴比例相等
    ax.set_aspect('equal')

    # 绘制 Hilbert 曲线
    ax.plot(x_coords, y_coords, 'b-', linewidth=2)

    # 添加起始点、终点和折点标记
    for x, y in points:

```

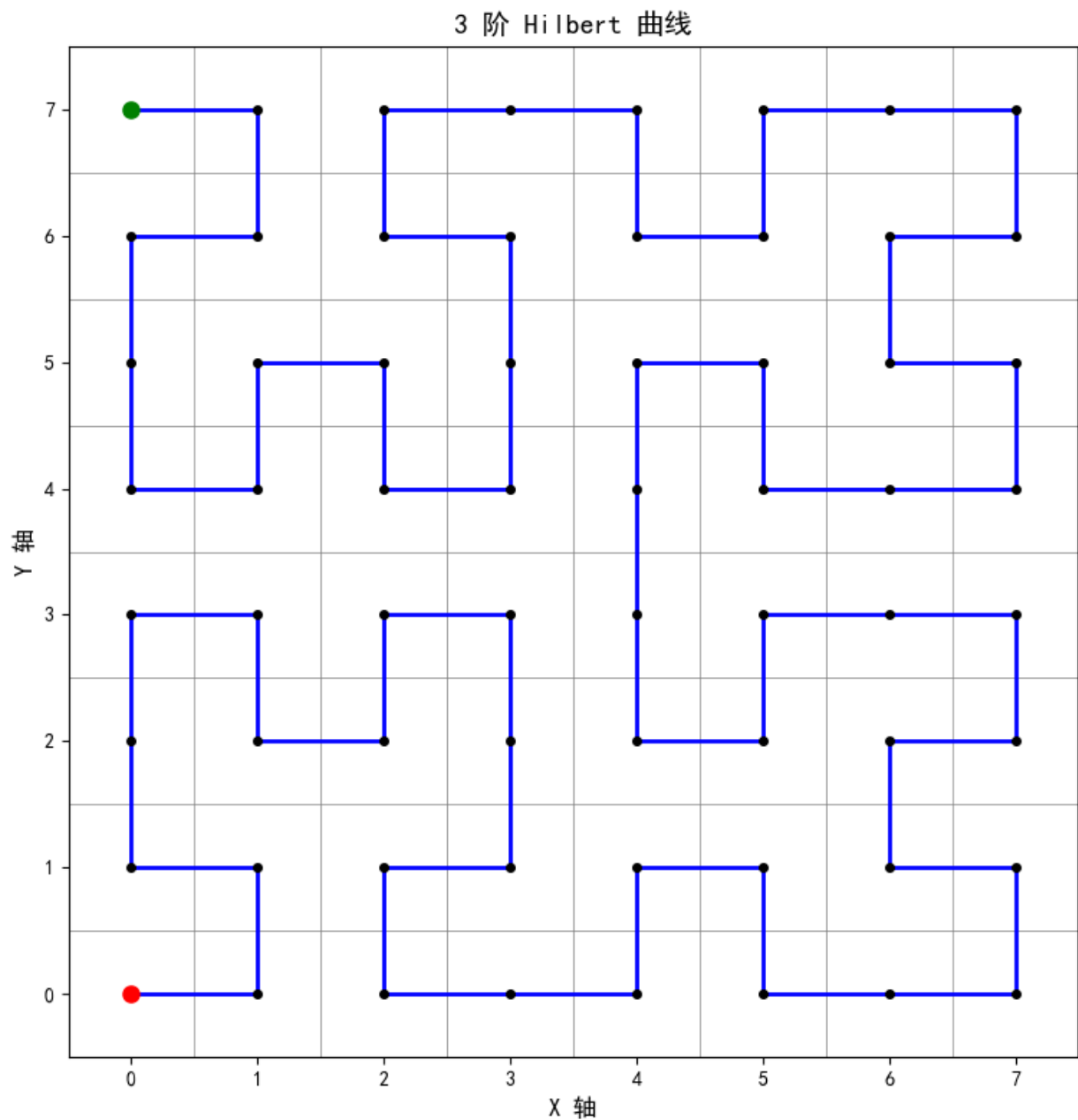
```

    ax.plot(x, y, 'ko', markersize=4) # 所有关键点 - 黑色
    ax.plot(x_coords[0], y_coords[0], 'ro', markersize=8) # 起点 - 红色
    ax.plot(x_coords[-1], y_coords[-1], 'go', markersize=8) # 终点 - 绿色

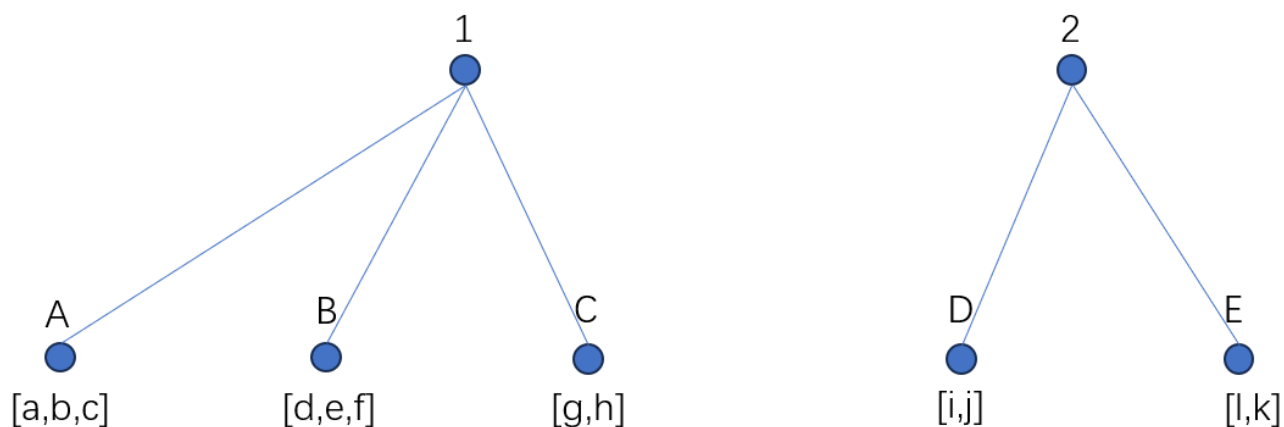
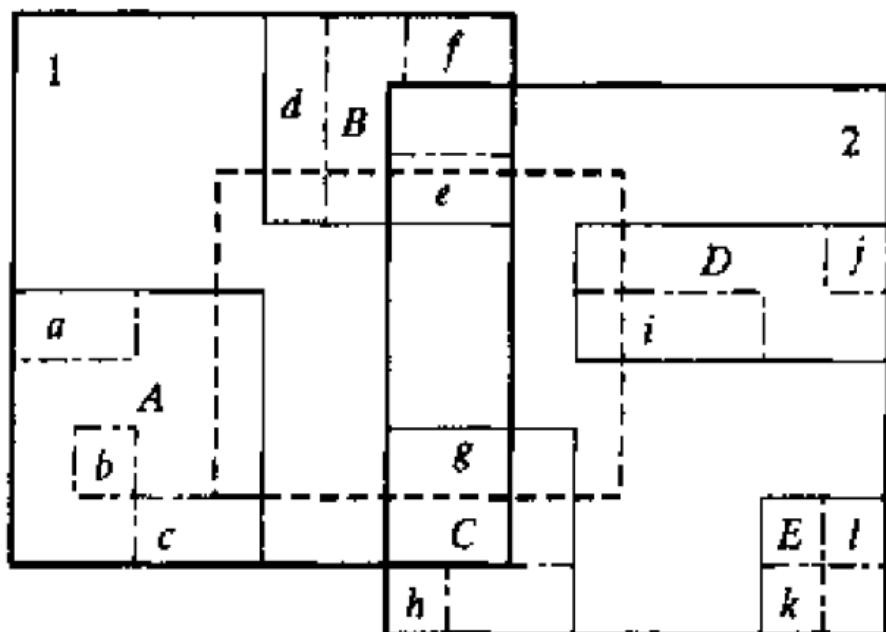
    return fig, ax

# 执行绘制
if __name__ == "__main__":
    fig, ax = plot_hilbert_with_grid(order=3, grid_size=8)
    plt.tight_layout()
    plt.show()

```



➤根据下图画出相应的R树，并写出虚线搜索框所要搜索的节点。叶节点为a, b,, l



虚线搜索框所要搜索的节点为d, e, i, g